

Чек-лист: принципы безопасного unsafe-кода

- 1 **Минимизация области unsafe.** Unsafe-блоки используются только в необходимых случаях.
- 2 **Гарантирование инвариантов.** Перед выполнением unsafe-операций установлено, что входные данные соответствуют ожидаемым:
 - Не возникает UB.
 - Соблюдены инварианты вызываемых функций (**Safety**-комментарий).
- 3 **Использование Safety-комментариев.** В комментариях объяснено, почему операция считается безопасной. Например:
// SAFETY: Указатель не null, память не освобождена до использования.
- 4 **Соблюдение правил алиасинга.** Не создано несколько изменяющих ссылок (**&mut**) на одну и ту же область памяти одновременно.
- 5 **Проверка указателей.** Перед разыменованием указателя установлено, что он не равен null и указывает на допустимую память. Для указателей, полученных через FFI или unsafe, использованы такие типы, как **NonNull<T>**, или применять дополнительные проверки.
- 6 **Избежание Use-After-Free.** Контролируется время жизни объектов. Данные не используются после их освобождения (use-after-free).
- 7 **Корректная работа с памятью.** Все операции с памятью в unsafe-блоках соблюдают дать её целостность. Это включает правильное использование операций, таких как **std::ptr::null** и **std::mem::forget**.
- 8 **Использование безопасных обёрток для unsafe-кода.** Когда возможно, unsafe-операции инкапсулированы в безопасных API, чтобы минимизировать их влияние на остальной код.
- 9 **Тестирование и код-ревью.** Для unsafe-кода проводится юнит-тестирование и проверки на границы памяти. Проводится ревью кода для обеспечения его безопасности.
- 10 **Работа с многозадачностью.** При использовании unsafe в многозадачных приложениях учитывается корректность работы с данными в разных потоках.